Lernskript: Nutzt unsere Spurerkennung benachbarte Fahrbahnmarkierungen?

Alles Wichtige zum XAI-Projekt – zum Lesen unterwegs

Brisild Capani
DHBW Stuttgart · DHBW Smart Rollerz · Modul Explainable AI

Stand: 27. Oktober 2026

Wie du das Skript liest. Die Kapitel bauen aufeinander auf: erst was das Modell tut (1–3), dann wie wir es untersuchen (4–6), dann was herauskam und wo die Grenzen liegen (7–8). Blaue Kästen fassen zusammen, orange Kästen warnen vor Denkfehlern. Am Ende stehen ein Glossar und Fragen, mit denen du prüfen kannst, ob du alles verstanden hast.

Inhaltsverzeichnis

1 Worum geht es überhaupt? 3

2 Was das Modell sieht und was es ausgibt 3

2.1 Die Eingabe . . . 3
2.2 Zeilenanker: die zentrale Idee von UFLD . . . 3
2.3 Die vier Ausgaben . . . 4
2.4 Von Punktzahlen zur Position . . . 4

3 Aufbau des Netzes 5

4 Das Verdeckungsexperiment 5

4.1 Die Idee . . . 5
4.2 Schritt für Schritt . . . 5
4.3 Drei Arten zu übermalen . . . 6
4.4 Die fünf Bedingungen . . . 7
4.5 Wie man die Bedingungen liest . . . 7
4.6 Was gemessen wird . . . 7

5 Grad-CAM und HiResCAM 8

5.1 Die Grundidee: Class Activation Maps . . . 8
5.2 Was wir erklären: die Zielgröße y . . . 8
5.3 Grad-CAM . . . 8
5.4 HiResCAM . . . 8

5.5 Warum der Unterschied bei UFLD wichtig ist 8
5.6 Die Auflösung ist das Hauptproblem 9
5.7 Wie wir Heatmap und Experiment vergleichen 9

6 Kurz erklärt: die Statistik **9**
6.1 Warum Median? 9
6.2 Gepaart: dasselbe Bild zweimal 9
6.3 Wilcoxon-Vorzeichen-Rang-Test 9
6.4 Was ein p-Wert bedeutet 9
6.5 Spearman-Korrelation 10

7 Die Übersichtsabbildung lesen **10**

8 Ergebnisse **11**
8.1 Altes Modell `bev`: 60 Bilder 11
8.2 2026er-Modell `bev_2026` 12
8.3 Erste Beobachtung zur Blendung 12

9 Grenzen und Denkfehler **12**
9.1 Wir verdecken, wo das Modell eine Linie vermutet 12
9.2 Weitere Grenzen 13
9.3 Typische Denkfehler 13

10 Wie es weitergeht **13**

11 Glossar **14**

12 Selbsttest **14**

1 Worum geht es überhaupt?

Unser Modellauto fährt autonom und muss dafür die Fahrbahnmarkierungen erkennen: die linke Linie, die gestrichelte Mittellinie und die rechte Linie. Das übernimmt ein neuronales Netz, eine Variante von *Ultra Fast Lane Detection v2* (UFLDv2).

Im echten Betrieb ist nicht immer jede Linie gut sichtbar: Sonnenlicht blendet, Schatten fallen, ein Hindernis steht auf der Linie, Markierungen sind abgenutzt. Trotzdem findet das Modell oft noch eine Linie. Die Frage ist: **woher weiß es, wo sie liegt?** Zwei Möglichkeiten:

1. Es nutzt die **sichtbaren Reste derselben Linie**, also die Stücke oberhalb und unterhalb der Lücke, und verbindet sie.
2. Es nutzt die **Nachbarlinien**: Liegt die Mittellinie hier, muss die rechte Linie ungefähr eine Spurbreite daneben liegen.

Das ist die **Forschungsfrage**: *Nutzt das Modell benachbarte Markierungen, um eine teilweise verdeckte Linie zu erkennen?*

Beantwortet wird sie mit zwei Werkzeugen aus der Explainable AI (XAI):

- **Verdeckungsexperimente:** Wir verändern das Bild gezielt und messen, wie sich die Vorhersage ändert. Das ist ein *kausaler* Test: Wir greifen ein und schauen, was passiert.
- **Grad-CAM und HiResCAM:** Diese Verfahren erzeugen Heatmaps, die zeigen sollen, welche Bildbereiche für eine Vorhersage wichtig waren. Sie greifen nicht ein, sondern werten nur die Gradienten des Netzes aus.

Der eigentliche XAI-Kern des Projekts ist der Vergleich: **Zeigen die Heatmaps dasselbe wie die Experimente?** Wenn ja, kann man den Heatmaps trauen. Wenn nein, wissen wir, dass sie für dieses Modell irreführend sein können.

Merke

Experimente beantworten *„Was passiert, wenn X fehlt?“*. Heatmaps beantworten *„Wohin zeigen die Gradienten?“*. Das ist nicht dasselbe. Unser Projekt prüft, wie gut das Zweite das Erste vorhersagt.

2 Was das Modell sieht und was es ausgibt

2.1 Die Eingabe

Die Kamera liefert ein Bild, das in eine Draufsicht umgerechnet wird (*Bird's Eye View*, BEV). Die Linien laufen darin ungefähr senkrecht von unten nach oben. Die schwarzen Dreiecke an den Rändern sind Bereiche ohne Bildinhalt, die bei der Umrechnung entstehen. Unten in der Mitte sieht man das eigene Fahrzeug.

Das Bild ist in Graustufen. Für das Netz wird es auf drei gleiche Kanäle kopiert, auf die Eingabegröße verkleinert und wie bei ImageNet normalisiert (Mittelwert abziehen, durch die Standardabweichung teilen).

2.2 Zeilenanker: die zentrale Idee von UFLD

Klassische Spurerkennung entscheidet für jeden einzelnen Pixel, ob er zu einer Linie gehört (*Segmentierung*). Das ist rechenaufwendig. UFLD macht es anders und deshalb schneller:

- Über das Bild werden **feste waagerechte Zeilen** gelegt, die **Zeilenanker** (beim alten Modell 35, beim 2026er 55).
- Jede Zeile wird in **Kästchen** (*Bins*) geteilt (50 bzw. 100).
- Für jede Zeile und jede der drei Linien gibt das Netz jedem Kästchen eine **Punktzahl** (Logit). Wo die Punktzahl am höchsten ist, liegt die Linie.
- Zusätzlich gibt es pro Zeile und Linie eine Ja/Nein-Ausgabe: **Gibt es die Linie in dieser Zeile überhaupt?**

Das Netz löst also lauter kleine *Klassifikationsaufgaben*: „In welchem Kästchen dieser Zeile liegt die Linie?“. Das Ergebnis ist **ein Punkt pro Zeile und Linie**, und die Punkte einer Linie ergeben zusammen die Linie im Bild.

Es gibt dasselbe auch mit senkrechten *Spaltenankern*, für Linien, die eher waagerecht verlaufen. In unseren Draufsichtbildern sind fast immer alle Zeilenanker aktiv und kaum Spaltenanker. Wir werten deshalb nur die Zeilenanker aus.

2.3 Die vier Ausgaben

Ausgabe	Form (altes Modell)	Bedeutung
`loc_row`	$50 \times 35 \times 3$	Punktzahl je Kästchen, Zeilenanker, Linie
`exist_row`	$2 \times 35 \times 3$	„Linie da?“ (nein/ja) je Zeilenanker und Linie
`loc_col`	$50 \times 35 \times 3$	dasselbe für Spaltenanker
`exist_col`	$2 \times 35 \times 3$	dasselbe für Spaltenanker

2.4 Von Punktzahlen zur Position

Aus den Punktzahlen einer Zeile wird die x-Position so berechnet wie im Code des Teams (`pred2coords`):

1. Suche das Kästchen b^* mit der höchsten Punktzahl.
2. Nimm dieses Kästchen und seine beiden Nachbarn b^*-1, b^*, b^*+1 und rechne ihre Punktzahlen mit Softmax in Wahrscheinlichkeiten p_i um.
3. Gewichteter Mittelwert: $\hat{b} = \sum_i p_i \cdot i$. Das ergibt eine Position *zwischen* den Kästchen, also feiner als das Raster.
4. In Pixel umrechnen: $x = (\hat{b} + 0{,}5) \,/\, (\text{Anzahl Kästchen} - 1) \cdot \text{Bildbreite}$.

Die y-Position ist fest durch den Anker vorgegeben: $y = \text{row_anchor}_k \cdot \text{Bildhöhe}$.

Merke

Das Modell beantwortet pro Zeile zwei Fragen: *„Wo ist die Linie?“* (welches Kästchen) und *„Ist sie da?“* (ja/nein). Alle unsere Messungen bauen genau darauf auf: Verschiebung der Position und Wahrscheinlichkeit für „ja“.

3 Aufbau des Netzes

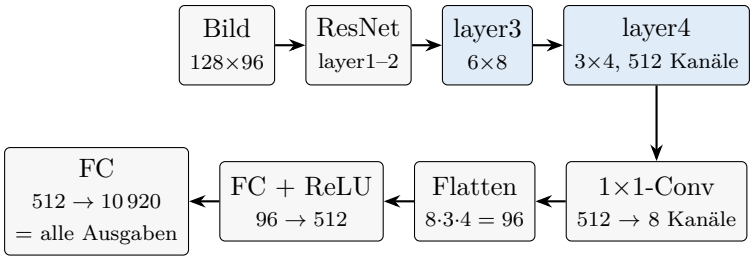


Abbildung 1: Aufbau des alten Modells `best_model_bev.pth`. Blau: Schichten, an denen wir die Heatmaps berechnen. Beim 2026er-Modell ist alles viermal so groß: Eingabe 512×384, layer4 mit 12×16 Zellen, Flatten auf 1536 Werte, Ausgabe 29 070 Werte.

Backbone. Ein verkleinertes ResNet (vom Team „9“ genannt, je Stufe ein Residual-Block) verarbeitet das Bild. Jede Stufe halbiert die Auflösung. Nach `layer4` ist aus dem 128×96-Bild ein Raster von nur 3×4 Zellen geworden, jede mit 512 Merkmalen.

Kopf. Eine 1×1-Faltung verdichtet die 512 Merkmale pro Zelle auf 8. Dann wird alles zu einem langen Vektor *plattgedrückt* (Flatten) und durch zwei voll verbundene Schichten (FC) geschickt. Die letzte gibt alle 10 920 Zahlen auf einmal aus: $50 \cdot 35 \cdot 3$ (`loc_row`) $+ 50 \cdot 35 \cdot 3$ (`loc_col`) $+ 2 \cdot 35 \cdot 3 + 2 \cdot 35 \cdot 3$ (`exist`).

Merke

Wegen Flatten + FC **kann jede Ausgabe von jeder Bildstelle abhängen**. Die Position der rechten Linie in Zeile 20 kann also durchaus von Pixeln an der Mittellinie beeinflusst werden. Ein Faltungsnetz mit begrenztem Sichtfeld könnte das nicht, UFLD kann es. Unsere Frage ist also berechtigt, und die Antwort ist nicht von vornherein klar.

Das ist auch der Grund, warum der Unterschied zwischen Grad-CAM und HiResCAM bei UFLD so wichtig ist (Kapitel 5).

Die zwei Modelle im Vergleich

	`bev` (Standard)	`bev_2026`
Datei	`best_model_bev.pth`	`best_model_bev_2026.pth`
Eingabe	128 × 96	512 × 384
Zeilenanker / Kästchen	35 / 50	55 / 100
Heatmap-Schicht	layer3, 6 × 8	layer4, 12 × 16
Laufzeit 60 Bilder (Batch)	wenige Minuten	ca. 1 Stunde

4 Das Verdeckungsexperiment

4.1 Die Idee

Wenn man wissen will, ob ein Modell eine Information *benutzt*, nimmt man sie weg und schaut, ob es schlechter wird. Das ist das Prinzip aller *Occlusion*- und *Ablation*-Methoden in der XAI. Wir verdecken also einzelne Linien und messen die Änderung.

4.2 Schritt für Schritt

1) **Bereich wählen.** Wir wählen die Zeilenanker zwischen 35 % und 65 % der Ankerliste aus (magenta). Beim 2026er-Modell sind das die Anker 19–34, beim alten die Anker 12–21. Weil die Anker fast das ganze Bild abdecken, entspricht das ungefähr dem mittleren Drittel des Bildes.

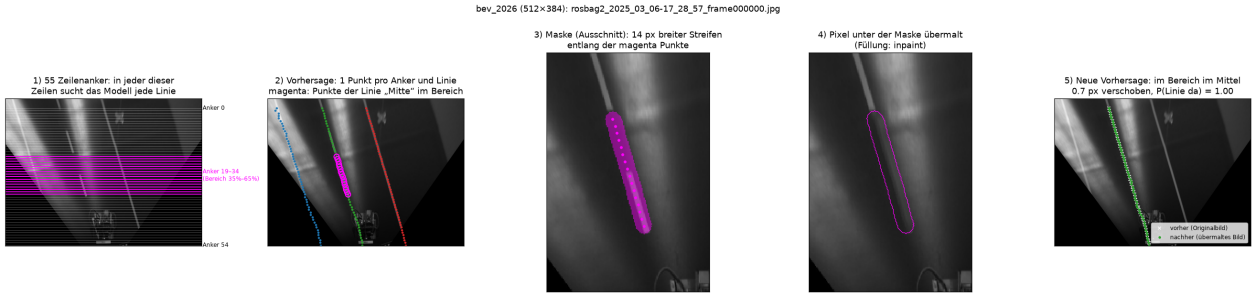


Abbildung 2: Das Experiment in fünf Schritten (2026er-Modell, Bild 0, Ziellinie Mitte).

2) **Punkte nehmen.** Das Modell sagt auf dem *Originalbild* voraus, wo die Linien liegen. Die Punkte der Ziellinie im gewählten Bereich (magenta Kreise) sind das Stück, das verschwinden soll.

3) **Maske zeichnen.** Wir verbinden diese Punkte zu einer Linie und machen sie dick: 14 px bei 512 px Bildbreite, 4 px bei 128 px. Diese Fläche heißt **Maske**. Sie liegt genau über der Markierung.

4) **Übermalen.** Alle Pixel unter der Maske werden ersetzt (siehe unten). Danach ist das Stück der Linie weg.

5) **Neu vorhersagen.** Das veränderte Bild geht noch einmal ins Modell. Wir vergleichen die neuen Punkte (grün) mit den alten (weiße ×).

4.3 Drei Arten zu übermalen

`inpaint` Die Lücke wird aus der Umgebung heraus aufgefüllt (OpenCV, Telea-Verfahren), wie der Reparaturpinsel in einer Bildbearbeitung. Am realistischsten und unser **Standard für Auswertungen**.

`median` Die Fläche wird mit dem typischen Grauwert der Straße daneben gefüllt. Einfach, aber es entstehen flache graue Balken, die im Schatten deutlich auffallen.

`blendung` Ein heller, weich auslaufender Fleck wie Sonne oder Scheinwerfer: Linie und Straße werden gleich hell. Das ist die Variante für die Frage nach dem *Lichteinfall*.

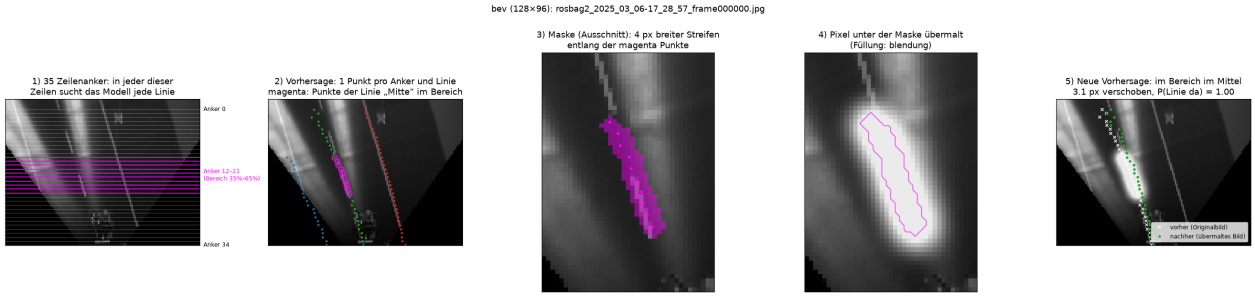


Abbildung 3: Dasselbe mit dem alten Modell und `blendung`. Man sieht, wie grob 128 × 96 Pixel sind (Schritt 3), und wie der Lichtfleck die Linie verschluckt (Schritt 4).

Achtung

Das Übermalen selbst verändert das Bild, und zwar nicht nur „Linie weg". Ein grauer Balken oder ein heller Fleck kann für das Modell selbst wie eine Struktur aussehen. Deshalb brauchen wir eine **Kontrollbedingung** (D) und prüfen jede Aussage mit mindestens zwei Füllungen.

4.4 Die fünf Bedingungen

Für eine **Ziellinie T** und eine **Nachbarlinie N**:

	verdeckt	Frage
0	nichts	Referenz, damit vergleichen wir alles
A	Stück von T	Wie gut schätzt das Modell die Lücke?
B	Stück von T + ganz N	Wird es schlechter, wenn auch der Nachbar fehlt?
C	nur N	Stört das Fehlen von N die *voll sichtbare* Linie T?
D	Stück von T + Streifen auf freier Straße	Kontrolle: Schadet *jede* gleich große Änderung?

Der Kontrollstreifen in D hat dieselbe Form wie N, liegt aber genau in der Mitte zwischen T und N, also auf freier Straße. D verändert damit ungefähr gleich viel Bildfläche wie B, nimmt aber keine Linie weg.

4.5 Wie man die Bedingungen liest

- **B deutlich schlechter als A** ⇒ das Modell hat N genutzt.
- **B auch schlechter als D** ⇒ das liegt wirklich an N und nicht daran, dass mehr vom Bild verändert wurde.
- **C schon groß** ⇒ das Modell hängt *immer* an N, auch wenn T selbst sichtbar ist. Das spricht für eine **gelernte Spurbreite** und nicht für ein gezieltes „Lücke mit Hilfe des Nachbarn füllen“.

Die letzte Unterscheidung ist feiner, als sie aussieht. Wenn B ungefähr A + C ist, addieren sich zwei unabhängige Effekte: „T hat eine Lücke“ und „N fehlt“. Wäre B deutlich *größer* als A + C, dann bräuchte das Modell N gerade *wegen* der Lücke, und das wäre die eigentliche Antwort auf die Forschungsfrage. Formal ist das der **Interaktionseffekt** $(B - A) - (C - 0)$.

4.6 Was gemessen wird

Gemessen wird nur an den Ankern des verdeckten Stücks von T:

`dx_mittel_px` mittlerer Abstand der neuen x-Position zur Referenz in Pixeln der Modellauflösung

`dx_mittel_pct` dasselbe in **% der Bildbreite**. Nur damit lassen sich beide Modelle vergleichen, denn 1 px bei 128 px Breite entspricht 4 px bei 512 px.

`P_exist_T` Wahrscheinlichkeit für „Linie da“, gemittelt über die Anker

`Anker_verloren` Anker, an denen das Modell die Linie gar nicht mehr findet

Achtung

Die Referenz ist die Vorhersage auf dem Originalbild, nicht die Wahrheit. Labels gibt es keine. Wir messen also, wie stark sich das Modell durch die Verdeckung *ändert*, nicht, ob es richtig liegt. Liegt das Modell schon im Original daneben, messen wir die Änderung einer falschen Vorhersage.

5 Grad-CAM und HiResCAM

5.1 Die Grundidee: Class Activation Maps

Die letzte Faltungsschicht (bei uns `layer4` oder `layer3`) hat für jede Bildzelle (i, j) und jeden Kanal k einen Wert A^k_{ij}, die *Aktivierung*. Grob gesagt: Wie stark „sieht“ Kanal k an dieser Stelle sein Muster, etwa eine helle senkrechte Kante?
Eine *Class Activation Map* (CAM) kombiniert diese Kanäle zu *einer* Karte: Kanäle, die für die erklärte Ausgabe y wichtig sind, bekommen viel Gewicht. Wie „wichtig“ bestimmt wird, unterscheidet die Verfahren. Beide nutzen dazu den **Gradienten** $\partial y / \partial A^k_{ij}$: Wie stark ändert sich y, wenn sich die Aktivierung an dieser Stelle ein wenig ändert?

5.2 Was wir erklären: die Zielgröße y

Eine CAM erklärt immer *eine* Zahl. Bei der Bildklassifikation ist das die Punktzahl einer Klasse („Hund“). Bei uns gibt es Tausende Ausgaben, also müssen wir wählen. Wir nehmen

$$y \;=\; \sum_{k \in \text{verdeckt}} z_{\,b^*_k,\,k,\,T}$$

also die Summe der Punktzahlen z, die das Modell *im Originalbild* den Gewinner-Kästchen b^*_k der verdeckten Anker k von Linie T gegeben hat. Im Code heißt das `RowBinTarget`.
Übersetzt fragt die Heatmap: **„Welche Bildbereiche halten das Stück von T an seiner ursprünglichen Stelle?“** Das passt genau zum Experiment, das misst, ob das Stück nach der Verdeckung woanders landet.

5.3 Grad-CAM

Grad-CAM [2] gibt jedem Kanal *ein* Gewicht, nämlich den über alle Bildstellen gemittelten Gradienten:

$$\alpha_k = \frac{1}{Z} \sum_{i,j} \frac{\partial y}{\partial A^k_{ij}}, \qquad L^{\text{Grad-CAM}}_{ij} = \text{ReLU}\Big(\sum_k \alpha_k \, A^k_{ij} \Big).$$

Dabei ist Z die Anzahl der Zellen. Die ReLU wirft negative Werte weg, denn gezeigt werden soll nur, was y *erhöht*.

5.4 HiResCAM

HiResCAM [3] mittelt nicht, sondern multipliziert Gradient und Aktivierung **Zelle für Zelle**:

$$L^{\text{HiResCAM}}_{ij} = \text{ReLU}\Big(\sum_k \frac{\partial y}{\partial A^k_{ij}} \, A^k_{ij} \Big).$$

5.5 Warum der Unterschied bei UFLD wichtig ist

Bei klassischen Klassifikationsnetzen folgt auf die letzte Faltung ein *Global Average Pooling*: Alle Zellen werden gemittelt, danach kommt eine lineare Schicht. Dort ist der Gradient für alle Zellen eines Kanals gleich, das Mitteln in Grad-CAM verliert also nichts, und beide Verfahren liefern dieselbe Karte.
UFLD hat aber **Flatten + FC** (Kapitel 3). Jede Zelle hat ihr *eigenes* Gewicht in der FC-Schicht. Der Gradient an der Mittellinie kann also ganz anders sein als der Gradient am Bildrand. Grad-CAM mittelt diese Unterschiede weg: Ein Kanal, der an der Mittellinie wichtig ist, bekommt sein Gewicht über das ganze Bild verteilt. Die Karte zeigt dann auch Stellen, an denen der Kanal zwar aktiv ist, die aber für y keine Rolle spielen.
HiResCAM behält die Unterschiede. Für ein Netz, das nach der letzten Faltung nur *eine* lineare Schicht hat, ist beweisbar, dass HiResCAM genau zeigt, welche Stellen zu y beitragen [3]. UFLD hat

zwei FC-Schichten mit ReLU dazwischen. Die Garantie gilt dann nicht streng, HiResCAM ist aber eine lokale, positionsgenaue Näherung und deutlich treuer als Grad-CAM.

Merke

Grad-CAM: ein Gewicht pro Kanal, über das Bild gemittelt ⇒ verwischt bei Flatten+FC-Köpfen.
HiResCAM: Gewicht pro Kanal *und* Stelle ⇒ positionsgenau.
Bei UFLD erwarten wir deshalb, dass HiResCAM besser zu den Experimenten passt. Genau das prüfen wir.

5.6 Die Auflösung ist das Hauptproblem

Die Karte hat nur so viele Zellen wie die Schicht: 6×8 beim alten Modell (layer3), 12×16 beim 2026er (layer4). Für die Darstellung wird sie auf Bildgröße hochskaliert und weichgezeichnet, das sieht feiner aus, als es ist. Beim alten Modell ist eine Zelle etwa 16 px breit, mehr als der Abstand zwischen einer Linie und der Straße daneben. Man kann damit sagen „das Modell schaut in Richtung Mittellinie“, aber nicht „genau auf die Markierung“.

5.7 Wie wir Heatmap und Experiment vergleichen

`N_Anteil` Welcher Anteil der gesamten Heatmap-Masse liegt auf der Maske von N?

`N_vs_Flaeche` `N_Anteil` geteilt durch den Flächenanteil der Maske am Bild. Ein Wert über 1 heißt: Die Heatmap liegt dort häufiger, als es die Größe der Fläche erwarten lässt. 1 wäre „zufällig verteilt“.

Spearman ρ Wir berechnen über alle Bilder, wie gut `N_Anteil` mit dem gemessenen Effekt $B - A$ zusammenhängt. Eine treue Heatmap müsste in genau den Bildern mehr auf N zeigen, in denen N tatsächlich hilft.

6 Kurz erklärt: die Statistik

6.1 Warum Median?

Die Verschiebungen sind schief verteilt: Meist ist es wenig, manchmal sehr viel. Der Mittelwert würde von wenigen Ausreißern bestimmt. Der Median, also der mittlere Wert, wenn man alle sortiert, beschreibt das typische Bild besser.

6.2 Gepaart: dasselbe Bild zweimal

Für jedes Bild haben wir A *und* B. Wir vergleichen also nicht zwei Gruppen von Bildern, sondern *dasselbe Bild unter zwei Bedingungen*. Das ist viel empfindlicher, weil Unterschiede zwischen den Bildern (hell, dunkel, Kurve, gerade) herausfallen. Pro Bild rechnen wir die Differenz $d = \mathrm{dx}_B - \mathrm{dx}_A$.

6.3 Wilcoxon-Vorzeichen-Rang-Test

Frage: Sind die Differenzen d *systematisch* positiv, oder schwanken sie nur zufällig um null? Der Test sortiert die $|d|$ nach Größe, vergibt Ränge und vergleicht die Rangsumme der positiven mit der der negativen Differenzen. Er setzt keine Normalverteilung voraus. Wir testen *einseitig*, denn die Hypothese hat eine Richtung: „Ohne N wird es *schlechter*.“

6.4 Was ein p-Wert bedeutet

Der p-Wert ist die Wahrscheinlichkeit, ein mindestens so deutliches Ergebnis zu sehen, *wenn es in Wirklichkeit gar keinen Effekt gäbe*. $p = 3 \cdot 10^{-8}$ heißt: Ohne echten Effekt wäre so ein Ergebnis praktisch unmöglich.

Achtung

Ein kleiner p-Wert heißt *nicht*, dass der Effekt groß ist, sondern nur, dass er sehr regelmäßig auftritt. Unsere Effekte sind oft nur 1–3 % der Bildbreite, aber in fast jedem Bild gleich gerichtet. Außerdem stammen alle Bilder aus *einer* Fahrt, und benachbarte Bilder ähneln sich. Die Bilder sind also nicht wirklich unabhängig, und die p-Werte sind deshalb zu optimistisch. Wir nehmen nur jedes 20. Bild, um das abzumildern.

6.5 Spearman-Korrelation

Misst, ob zwei Größen *gemeinsam steigen*, ohne dass der Zusammenhang linear sein muss. Beide Größen werden in Ränge umgerechnet und dann korreliert. $\rho = +1$: perfekt gleichläufig; 0: kein Zusammenhang; -1: gegenläufig. Werte unter etwa 0,3 gelten als schwach.

7 Die Übersichtsabbildung lesen

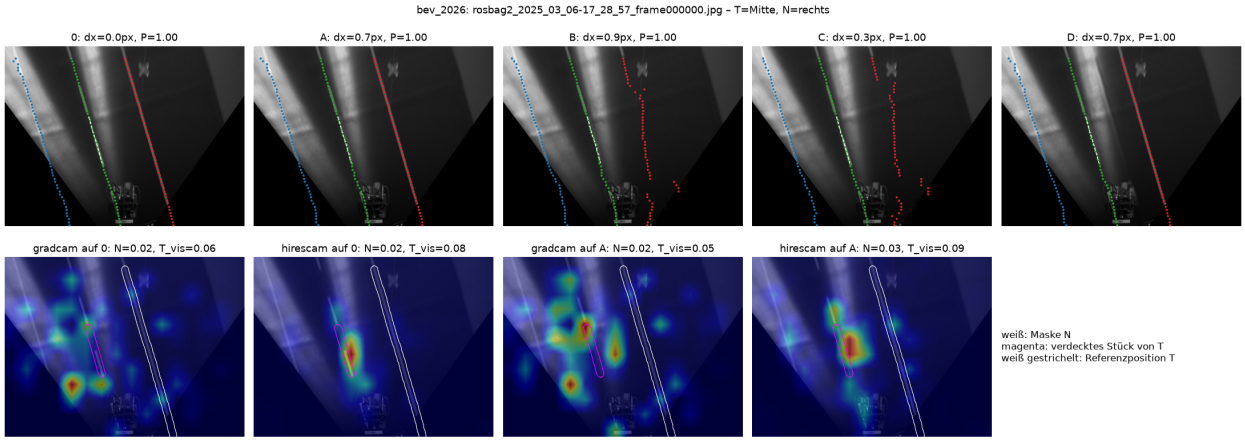


Abbildung 4: Ausgabe von `01_first_look.py`: 2026er-Modell, Bild 0, Ziellinie Mitte, Nachbar rechts, Füllung `inpaint`.

Obere Reihe: die fünf Bedingungen. Punkte: blau links, grün Mitte, rot rechts. Weiß gestrichelt ist die Referenzposition des verdeckten Stücks. Im Titel stehen Verschiebung und Sicherheit.
Untere Reihe: die Heatmaps für das verdeckte Stück von T, auf dem Originalbild (0) und auf Bild A. Rot heißt wichtig. Weiß umrandet ist die Nachbarlinie, magenta das verdeckte Stück. `N=0.02` heißt: 2 % der Karte liegen auf der Nachbarlinie.
Was man hier sieht:

- Die Mittellinie wird auch ohne die rechte Linie fast unverändert gefunden (B kaum schlechter als A). Das 2026er-Modell braucht den Nachbarn hier nicht.
- **HiResCAM** konzentriert sich auf die Mittellinie selbst, Grad-CAM verteilt sich über das halbe Bild. Das ist genau der in Kapitel 5 beschriebene Effekt.
- Interessant: HiResCAM zeigt auf Bild A auf die *übermalte* Stelle. Entweder nutzt das Modell dort noch Reste, oder die Karte ist zu grob, um die Lücke vom Rest der Linie zu trennen. Das ist offen.

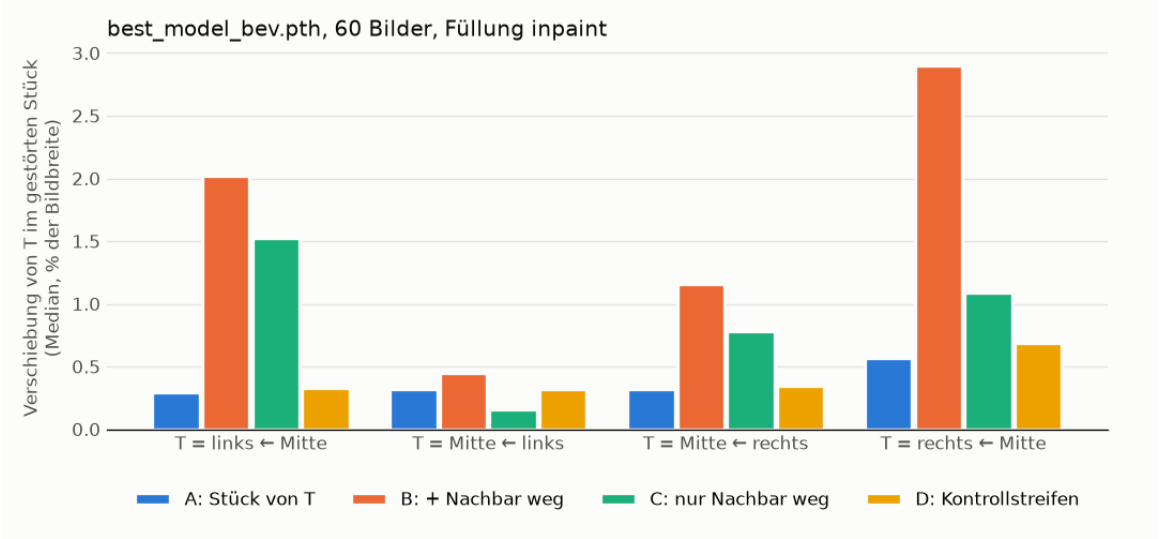


Abbildung 5: Verschiebung des verdeckten Stücks von T (Median über die Bilder, % der Bildbreite), Füllung `inpaint`. „T ← N“: Ziellinie T mit Nachbar N.

8 Ergebnisse

8.1 Altes Modell `bev`: 60 Bilder

T ← N	Bilder	A	B	C	D	p ($B>A$)	p ($B>D$)
links ← Mitte	50	0,29	**2,02**	1,52	0,33	$3 \cdot 10^{-8}$	$1 \cdot 10^{-6}$
Mitte ← links	56	0,32	0,45	0,16	0,32	$2 \cdot 10^{-4}$	0,009
Mitte ← rechts	58	0,32	**1,16**	0,78	0,35	$3 \cdot 10^{-10}$	$1 \cdot 10^{-8}$
rechts ← Mitte	59	0,57	**2,91**	1,09	0,69	$8 \cdot 10^{-11}$	$1 \cdot 10^{-9}$

1. Das alte Modell nutzt die Nachbarlinien. In allen vier Fällen ist B signifikant schlechter als A *und* als die Kontrolle D. Füll-Artefakte erklären das also nicht. Am stärksten hängen die **Außenlinien an der Mittellinie**. Die Mitte hängt mehr an der rechten als an der linken Linie, denn links fehlt oft eine echte Markierung.
2. Es ist vor allem ein allgemeiner Effekt. Schon C, bei dem T ganz sichtbar bleibt, verschiebt T deutlich. Bei „links ← Mitte“ ist B (2,02) etwa A + C (0,29 + 1,52 = 1,81). Die Effekte addieren sich also. Das Modell scheint eine **Spurbreite gelernt** zu haben und setzt die Linien relativ zueinander. Dass es N *speziell zum Füllen der Lücke* nutzt, lässt sich daraus nicht ablesen.
3. Die Effekte sind klein, aber regelmäßig. 1–3 % der Bildbreite sind bei 128 px nur 1–4 px. In 88–95 % der Bilder geht der Effekt aber in dieselbe Richtung.
4. Die Heatmaps passen nur teilweise.

- Bei „links ← Mitte“ zeigt nur **HiResCAM** überdurchschnittlich auf die Mittellinie, das 1,6-Fache des Flächenanteils. Grad-CAM zeigt fast nichts, das 0,09-Fache. Hier stimmt nur HiResCAM mit dem Experiment überein.
- Bei „Mitte ← links“ ist der Effekt klein, und beide Heatmaps zeigen fast nichts auf N. Das ist stimmig.
- Bei „rechts ← Mitte“ ist der Effekt am größten, aber beide Heatmaps liegen nur etwa beim Flächenanteil. Hier passen die Erklärungen nicht.
- Pro Bild hängen Heatmap-Anteil und Effekt nur schwach zusammen (ρ zwischen $-0{,}25$ und $+0{,}45$). Die Heatmaps sagen nicht verlässlich voraus, *in welchem Bild* der Nachbar wichtig ist. Bei 6×8 Zellen ist das aber auch kaum zu erwarten.

8.2 2026er-Modell `bev_2026`

Der Batch-Lauf mit dem 2026er-Modell war bei Erstellung dieses Skripts noch nicht fertig. Die Einzelbilder deuten darauf hin, dass es bei gut sichtbaren Linien weniger am Nachbarn hängt (Abb. 4), bei fehlenden Linien aber stark, und dann eher die Position verschiebt als unsicher zu werden.

8.3 Erste Beobachtung zur Blendung

Auf einem einzelnen Bild, also noch ohne Statistik: Liegt ein Lichtfleck auf der freien Straße *zwischen* zwei Linien (Bedingung D mit `blendung`), verschiebt das alte Modell die Linie um fast **20 % der Bildbreite**. Das Modell hält den hellen Fleck offenbar selbst für eine Linie. Für das Fahrzeug ist das wahrscheinlich gefährlicher als eine verdeckte Linie, denn das Modell ist sich dabei weiter sicher.

9 Grenzen und Denkfehler

9.1 Wir verdecken, wo das Modell eine Linie vermutet

Ohne Labels folgen die Masken den *Vorhersagen*. Bei gut sichtbaren Linien ist das egal, aber nicht überall:

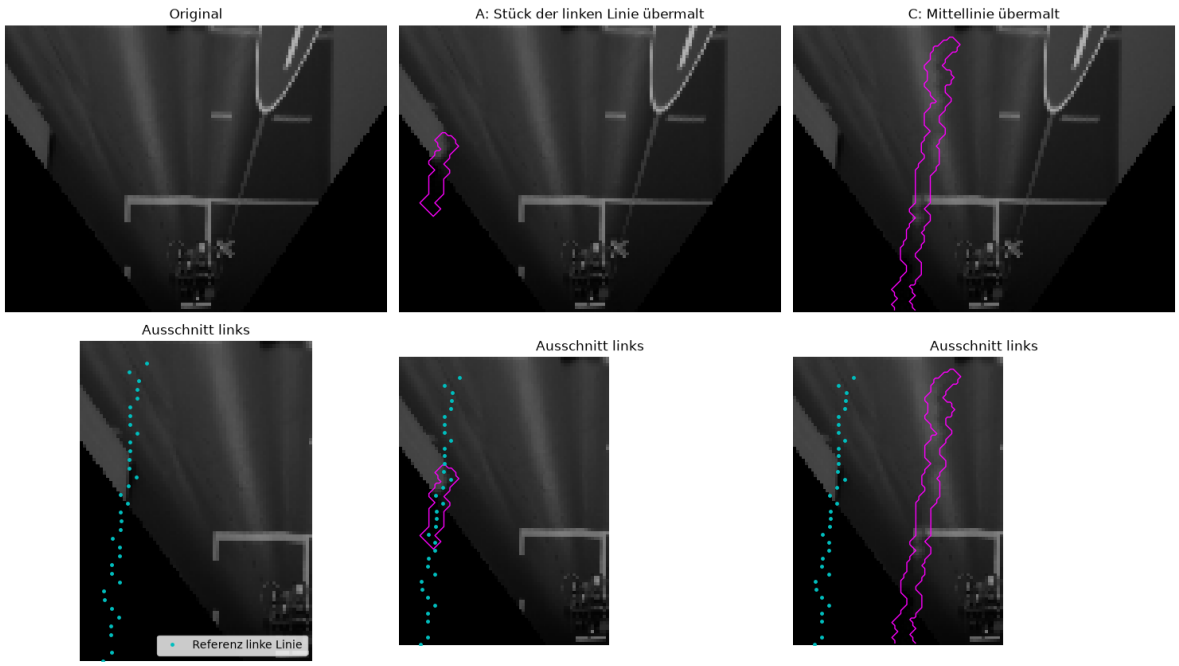


Abbildung 6: Bild 600, altes Modell: was tatsächlich übermalt wird. Oben das ganze Bild, unten der linke Ausschnitt. Türkis die „linke Linie“ des Modells.

- **Links ist keine Markierung.** Die „linke Linie“ des Modells läuft oben an einer Bordsteinkante und unten am schwarzen Rand des Bildausschnitts entlang. Die Maske in A übermalt also nichts Relevantes, und dass dort kaum etwas passiert, ist kein Befund.
- **Das Bild zeigt eine Kreuzung.** Die Maske der „Mittellinie“ (C) liegt auf Straße und schneidet das linke Ende der Haltelinie ab. Dass die Sicherheit für links dann von 0,86 auf 0,59 fällt, liegt vermutlich an der fehlenden Haltelinien-Ecke und nicht an einer fehlenden Mittellinie.

Dieses Bild zeigt also etwas anderes als die Forschungsfrage: An Kreuzungen orientiert sich das Modell an markanten Strukturen wie der Haltelinie, und als „linke Linie“ nimmt es notfalls eine Kante. Auch das ist ein XAI-Befund, gehört aber getrennt ausgewertet.

9.2 Weitere Grenzen

- **Keine Labels:** Wir messen Änderung, nicht Fehler (Kapitel 4).
- **Eine Fahrt:** Alle 1192 Testbilder stammen aus einer Aufnahme vom 6. 3. 2025. Andere Strecken oder Licht können anders aussehen, und die Bilder sind nicht unabhängig.
- **Füll-Artefakte:** Jede Verdeckung erzeugt etwas Neues im Bild. D und mehrere Füllungen mildern das ab, schließen es aber nicht ganz aus.
- **Grobe Heatmaps:** 6×8 bzw. 12×16 Zellen für Linien, die nur wenige Pixel breit sind.
- **Nur Zeilenanker:** Spaltenanker werten wir nicht aus. In den Draufsichtbildern spielen sie kaum eine Rolle.
- **Ankerformel:** Ob `row_anchor = linspace(100, 1540, num_row)/1550` wirklich die Formel aus dem Training ist, muss das Team bestätigen. Davon hängt die Zuordnung Anker $\leftrightarrow$ Bildzeile ab.

9.3 Typische Denkfehler

Achtung

- *„Die Heatmap ist rot an der Mittellinie, also nutzt das Modell die Mittellinie.“* Nein, die Heatmap zeigt Gradienten. Ob das Modell die Stelle *braucht*, zeigt erst das Experiment.
- *„p ist winzig, also ist der Effekt wichtig.“* Nein, er ist regelmäßig. Wie groß er ist, zeigen die Prozentwerte.
- *„B ist größer als A, also füllt das Modell die Lücke mit dem Nachbarn.“* Erst wenn B auch größer als A + C ist. Sonst wirkt der Nachbar einfach immer.
- *„In Bild X passiert nichts, also ist das Modell robust.“* Erst prüfen, ob dort überhaupt eine echte Linie verdeckt wurde.

10 Wie es weitergeht

1. **Sichtbarkeitsprüfung:** pro Linie und Bild prüfen, ob die Linie unter der Maske deutlich heller ist als die Straße daneben. Dann die Statistik getrennt für „sichtbar“ und „geraten“ auswerten.
2. **Kreuzungen** mit Haltelinie und Abbiegungen markieren oder ausschließen.
3. **Interaktion** $(B - A) - C$ als eigene Kennzahl: die direkte Antwort auf die Forschungsfrage.
4. **Batch mit `blendung`:** Robustheit gegen Lichteinfall mit Zahlen belegen.
5. **Ausblick fürs Fahrzeug:** Mit künstlichen Lichtflecken im Training (*Data Augmentation*) könnte das Modell lernen, Blendung zu ignorieren. Mit unseren Skripten lässt sich dann vorher und nachher vergleichen, ob es wirklich robuster geworden ist.

Die Skripte auf einen Blick

Skript	Zweck
`00_check_setup.py`	Lädt das Modell, prüft Ausgaben und Gradienten
`01_first_look.py`	Ein Bild: alle Bedingungen + Heatmaps als Abbildung
`02_batch.py`	Viele Bilder + Statistik (`-resume` zum Fortsetzen)
`03_schritte.py`	Schritt-für-Schritt-Abbildung wie in Abb. 2
`04_doku_bilder.py`	Bilder für Doku und dieses Skript neu erzeugen
`interaktiv.ipynb`	Regler: Bild, Linie, Bedingung, Füllung selbst ausprobieren

Alle mit `.venv/bin/python experiments/<skript>` im Projektordner, Modell wählen mit `-model bev` oder `-model bev_2026`.

11 Glossar

Ablation / Occlusion Teile der Eingabe entfernen oder verdecken, um ihre Wirkung zu messen.

Aktivierung A_{ij}^k Wert von Kanal k an Stelle (i, j) einer Schicht.

BEV Bird's Eye View, Draufsicht.

Bin / Kästchen Abschnitt einer Ankerzeile. Das Netz klassifiziert, in welchem Bin die Linie liegt.

CAM Class Activation Map, Heatmap aus Aktivierungen und Gewichten.

FC-Schicht Fully Connected: Jeder Eingang ist mit jedem Ausgang verbunden.

Flatten Ein Raster zu einem langen Vektor plattdrücken. Dabei bleibt die Position als „welcher Eintrag" erhalten.

Global Average Pooling Mittelwert über alle Stellen eines Kanals. UFLD hat das *nicht*.

Gradient Wie stark sich eine Ausgabe ändert, wenn sich ein Wert leicht ändert.

Inpainting Bildbereich aus der Umgebung heraus auffüllen.

Interaktionseffekt Wirkung, die nur in der *Kombination* zweier Eingriffe auftritt: $(B - A) - C$.

Logit Rohe Punktzahl vor der Softmax.

Maske Fläche, die übermalt wird.

Median Mittlerer Wert der sortierten Werte, robust gegen Ausreißer.

p-Wert Wahrscheinlichkeit für ein so deutliches Ergebnis ohne echten Effekt.

Referenz (Bedingung 0) Vorhersage auf dem unveränderten Bild.

ReLU $\max(0, x)$: negative Werte werden zu 0.

Softmax Wandelt Punktzahlen in Wahrscheinlichkeiten um, die sich zu 1 summieren.

Spearman ρ Rangkorrelation: Steigen zwei Größen gemeinsam?

Treue (Faithfulness) Wie gut eine Erklärung das tatsächliche Verhalten des Modells wiedergibt.

Wilcoxon-Test Prüft, ob gepaarte Differenzen systematisch in eine Richtung gehen.

Zeilenanker Feste Bildzeile, in der das Modell jede Linie sucht.

12 Selbsttest

Versuch die Fragen erst selbst zu beantworten, bevor du die Antwort liest.

1. *Was gibt UFLD pro Zeilenanker aus?*
   Für jede der 3 Linien eine Punktzahl pro Kästchen (wo ist die Linie?) und eine Ja/Nein-Ausgabe (ist sie da?).
2. *Warum kann bei UFLD die Position der rechten Linie von Pixeln an der Mittellinie abhängen?*
   Weil der Kopf Flatten + FC nutzt: Jede Ausgabe ist mit jeder Bildzelle verbunden.
3. *Was ist der Unterschied zwischen Bedingung B und D, und warum brauchen wir D?*
   B nimmt die Nachbarlinie weg, D verändert eine ähnlich große Fläche auf freier Straße. D zeigt, ob jede Bildänderung schadet. Nur wenn B schlechter ist als D, liegt es am Nachbarn.

4. *C ist groß und $B \approx A + C$. Was folgt daraus?*
   Der Nachbar wirkt immer, unabhängig von der Lücke. Das spricht für eine gelernte Spurbreite, nicht für gezieltes Lückenfüllen.

5. *Warum mittelt Grad-CAM bei UFLD Information weg?*
   Es gibt jedem Kanal ein über alle Stellen gemitteltes Gewicht. Bei Flatten + FC ist der Gradient aber an jeder Stelle anders.

6. *Was genau erklärt unsere Heatmap?*
   Die Summe der Punktzahlen der Referenz-Kästchen an den verdeckten Ankern von T, also „was hält dieses Stück an seiner Stelle?“.

7. *$p = 10^{-10}$, Effekt 1 % der Bildbreite. Großer oder kleiner Effekt?*
   Klein, aber sehr regelmäßig. Der p-Wert sagt nichts über die Größe.

8. *Warum ist Bild 600 kein gutes Beispiel für die Forschungsfrage?*
   Links gibt es keine echte Markierung, und die „Mittellinie“ liegt an einer Kreuzung. Die Masken übermalen Kante, Schwarz und die Haltelinien-Ecke statt echter Linien.

9. *Warum vergleichen wir die Modelle in % der Bildbreite?*
   Weil sie verschieden große Bilder sehen: 1 px bei 128 px Breite entspricht 4 px bei 512 px.

10. *Was wäre die direkte Antwort auf die Forschungsfrage, und wie misst man sie?*
    Ob der Nachbar *gerade wegen* der Lücke hilft: Interaktionseffekt $(B - A) - C$, getrennt für Bilder mit wirklich sichtbaren Linien.

Literatur

[1] Z. Qin, P. Zhang, X. Li: *Ultra Fast Deep Lane Detection With Hybrid Anchor Driven Ordinal Classification.* IEEE TPAMI, 2022. (UFLDv2)

[2] R. R. Selvaraju et al.: *Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization.* ICCV, 2017.

[3] R. L. Draelos, L. Carin: *Use HiResCAM instead of Grad-CAM for faithful explanations of convolutional neural networks.* arXiv:2011.08891, 2020.